Unity Graphics Debug Bridge (UGDB) — 개발 계획서 v3

> RenderDoc에서 복사 → UGDB에 붙여넣기 → Unity 에셋 역추적

---

1. 프로젝트 개요

1.1 문제 정의

RenderDoc에서 드로우콜을 디버깅할 때 가장 불편한 점:

- 셰이더: 바이너리/해시로 보임 → "이게 유니티에서 어떤 .shader 파일이지?"
- 텍스처: Slot 0에 2048x2048 ASTC_6x6이 바인딩됨 → "이게 어떤 에셋이지?"
- **Constant Buffer**: offset 0에 0.73이 들어있음 → "이게 _Metallic이야? _Smoothness야?"

현재는 이걸 사람이 머릿속으로 대조하거나, 유니티 에디터를 왔다갔다하면서 하나씩 찾아야 함.

1.2 목표

RenderDoc에서 복사 → UGDB에 붙여넣기 → Unity 에셋/프로퍼티를 바로 찾아주는 룩업 툴.

- Play 모드에서 Snap 버튼 → 씬의 모든 머티리얼-셰이더-텍스처-스칼라 체인을 인덱싱
- RenderDoc에서 아무 정보나 복사해서 붙여넣으면 파서가 자동 판별 + 검색
- 결과에서 Unity 에셋 경로, 프로퍼티 이름, 사용 오브젝트를 즉시 확인

1.3 타겟 환경

항목	사양
Unity 버전	Built-in RP (2021.3+)
플랫폼	Editor (Windows)
RenderDoc	1.26+
그래픽스 API	DX11 우선, Vulkan 확장 고려

2. 사용 워크플로우

기본 플로우: RenderDoc에서 복사 → UGDB에 Ctrl+V → Unity 에셋 확인

시나리오 1: "이 텍스처가 뭐지?"

```
1. RenderDoc Pipeline State → Pixel Shader Resources에서 텍스처 행 우클릭 → Copy
   클립보드에 복사됨:
   "2048x2048 BC7_UNORM Mips:12"
   또는 Resource 패널에서:
   "Texture2D 2048x2048 12 mips - ASTC_6x6"

2. UGDB 윈도우에서 Ctrl+V

3. 파서가 자동 판별: 텍스처 검색
   → 2048x2048, ASTC_6x6 (또는 BC7_UNORM) 추출

4. 결과:
   T_Hero_Body_D.png
   Assets/Textures/Hero/T_Hero_Body_D.png
   → _MainTex in M_Hero_Body (Custom/CharacterPBR)
   → Character_Hero (vtx:12450, idx:34560)
   [Ping Asset] [Copy Path] [Select GO]
```

시나리오 2: "이 드로우콜이 뭐지?"

```
1. RenderDoc Event Browser에서 드로우콜 우클릭 → Copy
   클립보드:
   "DrawIndexed(34560, 1, 0, 0, 0)"
   또는 API Inspector에서:
   "DrawIndexed()\n  IndexCount: 34560\n  InstanceCount: 1"

2. UGDB에 Ctrl+V

3. 파서가 자동 판별: 지오메트리 검색
   → indexCount: 34560 추출

4. 결과:
   Character_Hero (/MainStage/Characters/Hero)
   M_Hero_Body (Custom/CharacterPBR)
   vtx:12450 idx:34560
   [Select in Hierarchy] [Ping Material]
```

시나리오 3: "이 CB 값들이 어떤 프로퍼티지?"

```
1. RenderDoc Pipeline State → Constant Buffer 뷰에서 복사
   클립보드:
   "float4  1.000  0.950  0.900  1.000
    float   0.730
    float   1.000"
   또는:
   "    [0]: 1, 0.95, 0.9, 1
    [16]: 0.73
    [20]: 1"

2. UGDB에 Ctrl+V

3. 파서가 자동 판별: CB 검색
   → float 값들 추출: (1.0, 0.95, 0.9, 1.0), 0.73, 1.0

4. 결과:
   M_Hero_Body (Custom/CharacterPBR) 매칭
   _Color = (1.0, 0.95, 0.9, 1.0)       → CB offset 0
   _Metallic = 0.73                      → CB offset 16
   _BumpScale = 1.0                      → CB offset 20
   [Ping Material] [Select GO]
```

시나리오 4: Pipeline State 통째로 붙여넣기

```
1. RenderDoc Pipeline State 패널에서 여러 정보를 한번에 복사
   (텍스처 슬롯 테이블 전체, 또는 셰이더 + 리소스 정보)

2. UGDB에 Ctrl+V

3. 파서가 복합 정보 추출:
   → 텍스처: 2048x2048 ASTC_6x6 (slot 0), 2048x2048 ASTC_6x6 (slot 1)
   → CB: 0.73 (float)
   → Geometry: idx 34560

4. 복합 매칭 결과 (95% confidence):
   Character_Hero / M_Hero_Body
   Slot 0 = _MainTex → T_Hero_Body_D.png
   Slot 1 = _NormalMap → T_Hero_Body_N.png
   CB[16] = _Metallic (0.73)
```

---

3. 아키텍처

```
┌──────────────────────────────────────────────────────────┐
│                    UGDB Editor Window                    │
│  ┌──────────────────────────────────────────────────┐    │
│  │          Paste Area (Ctrl+V)                     │    │
│  │  "RenderDoc에서 복사한 내용을 붙여넣기"          │    │
│  └──────────────────────┬───────────────────────────┘    │
│                         │                                │
│  ┌──────────────────────▼───────────────────────────┐    │
│  │               Clipboard Parser                   │    │
│  │  텍스트 패턴 매칭 → 자동 판별 + 키 추출          │    │
│  └──────────────────────┬───────────────────────────┘    │
│                         │                                │
│  ┌──────────────────────▼───────────────────────────┐    │
│  │               Lookup Engine                      │    │
│  │  텍스처/셰이더/CB/지오메트리 인덱스 검색          │    │
│  └──────────────────────┬───────────────────────────┘    │
│                         │                                │
│  ┌──────────────────────▼───────────────────────────┐    │
│  │               Results Panel                      │    │
│  │  매칭 결과 + Unity 에셋 정보 + 액션 버튼         │    │
│  └──────────────────────────────────────────────────┘    │
│                         ▲                                │
│  ┌──────────────────────┴───────────────────────────┐    │
│  │           Scene Snapshot (인덱싱 데이터)         │    │
│  │  Play 모드에서 Snap → 씬 전체 체인 인덱싱        │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

핵심 모듈 (4개)

모듈	역할
SceneSnapshot	Play 모드에서 씬의 카메라/렌더러/라이트/머티리얼/텍스처/스칼라를 수집하여 검색 가능한 인덱스 생성
ClipboardParser	붙여넣은 텍스트에서 RenderDoc 포맷을 자동 판별하고 검색 키를 추출
LookupEngine	텍스처(해상도/포맷), 셰이더(이름/키워드), CB(값/타입), 지오메트리(vtx/idx) 기준 검색 + 복합 매칭
SearchWindow	붙여넣기 UI + 결과 표시 + Unity 에셋 연동 (Ping, Select, Copy)

---

4. 단계별 개발 계획

Phase 1 — Scene Snapshot + Lookup Engine (3~4주)

목표: Play 모드에서 Snap → 인덱싱 → 검색으로 Unity 에셋 찾기

1-1. Scene Snapshot: 수집 대상

Play 모드에서 버튼 클릭 시, 현재 씬에 살아있는 런타임 오브젝트를 기반으로 수집:

```
수집 흐름:
1. SceneManager로 로드된 씬 목록
2. Camera.allCameras로 활성 카메라 목록
3. 각 카메라의 프러스텀 컬링 → 보이는 Renderer 목록
4. 각 Renderer의 머티리얼-셰이더-텍스처-스칼라 풀체인

수집 데이터:
├── Renderer별
│   ├── GameObject 이름 + hierarchy 경로 + 소속 씬
│   ├── Mesh: vertexCount, indexCount, subMeshCount
│   ├── 보이는 카메라 목록
│   ├── lightmapIndex, lightProbeUsage, reflectionProbeUsage
│   └── Materials[] (아래 체인)
│
├── Material별
│   ├── 이름, instanceId, 인스턴스 여부
│   ├── renderQueue
│   └── Shader (아래 체인)
│
├── Shader별
│   ├── 이름 (예: "Custom/CharacterPBR")
│   ├── 에셋 경로
│   ├── passCount
│   ├── 활성 키워드 목록
│   └── 전체 프로퍼티 목록 (선언 순서대로):
│       ├── 프로퍼티 인덱스 (→ GPU 슬롯 순서 대응)
│       ├── 이름 + nameID
│       ├── 타입 (Texture/Float/Range/Color/Vector/Int)
│       └── 어트리뷰트 ([Normal], [HDR] 등)
│
├── Shader Variant 추적 (핵심)
│   ├── 머티리얼별 활성 키워드 조합 → variant 식별 키
│   │   예: "Custom/CharacterPBR|_EMISSION|_NORMALMAP"
│   ├── 텍스처 프로퍼티별 실제 사용 여부 판별:
│   │   ├── tex != null && tex != default → 실제 사용 (realSlot)
│   │   └── tex == null || tex == white/default → 미사용 (dummySlot)
```

```
│   │   (키워드 OFF 시 해당 텍스처가 null/default로 남는 것을 이용)
│   ├── variant별 실제 텍스처 슬롯 패턴:
│   │   예: Variant [_NORMALMAP, _EMISSION]
│   │       → slot 0: _MainTex (real), slot 1: _NormalMap (real), slot 2: _Emissi
│   │   예: Variant [_NORMALMAP]
│   │       → slot 0: _MainTex (real), slot 1: _NormalMap (real), slot 2: _Emissi
│   │   예: Variant [] (키워드 없음)
│   │       → slot 0: _MainTex (real), slot 1: _NormalMap (dummy), slot 2: _Emiss
│   └── 씬 전체에서 같은 셰이더의 variant 조합 목록 수집
│       → 어떤 variant가 실제로 사용 중인지 한눈에 파악
│
├── Texture 프로퍼티별
│   ├── 에셋 경로
│   ├── 해상도, 포맷, 밉맵 수
│   ├── filterMode, wrapMode
│   ├── 메모리 크기
│   └── 텍스처 타입 (Texture2D / RT / Cubemap)
│
├── Scalar 프로퍼티별
│   ├── Float/Color/Vector/Int 값
│   └── TextureOffset/Scale (_ST)
│
├── MaterialPropertyBlock 오버라이드 (있는 경우)
│   └── 오버라이드된 프로퍼티 + 값
│
├── Light별
│   ├── 타입, color, intensity, range
│   ├── shadow 설정 + shadowmap 해상도
│   └── cookie 텍스처
│
└── 글로벌 상태
    ├── 글로벌 텍스처 (_CameraDepthTexture 등)
    ├── 활성 RenderTexture 목록
    ├── 라이트맵 텍스처 목록
    └── RenderSettings, QualitySettings
```

1-2. Lookup Engine: 인덱스 구조

수집 데이터로부터 5개의 검색 인덱스를 빌드:

```
// 1. 텍스처 인덱스: 해상도+포맷 → 텍스처 목록
Dictionary<TextureSignature, List<TextureEntry>> textureIndex;

// 2. 셰이더 인덱스: 이름/키워드 → 셰이더+사용 머티리얼
Dictionary<string, ShaderEntry> shaderByName;
Dictionary<string, List<ShaderEntry>> shaderByKeyword;
```

```
// 3. CB 값 인덱스: float 값 → 프로퍼티
Dictionary<float, List<ScalarEntry>> scalarByValue;
Dictionary<Vector4, List<ScalarEntry>> colorByValue;

// 4. 지오메트리 인덱스: vtx+idx → 렌더러
Dictionary<(int vtx, int idx), List<RendererEntry>> geometryIndex;

// 5. Variant 인덱스: 슬롯 패턴 → variant 목록
//    RenderDoc에서 텍스처 슬롯을 붙여넣으면
//    "real 2개 + dummy 1개" 같은 패턴으로 variant를 특정
Dictionary<string/*variantKey*/, VariantEntry> variantIndex;
Dictionary<string/*slotPattern*/, List<VariantEntry>> variantBySlotPattern;
```

1-3. Shader Variant 추적 상세

```
public struct VariantEntry
{
    // variant 식별
    public string shaderName;         // "Custom/CharacterPBR"
    public string[] activeKeywords;   // ["_NORMALMAP", "_EMISSION"]
    public string variantKey;         // "Custom/CharacterPBR|_EMISSION|_NORMALMA

    // 이 variant를 사용하는 머티리얼 목록
    public List<MaterialEntry> materials;

    // 텍스처 슬롯 패턴 (RenderDoc 매칭의 핵심)
    public List<SlotState> textureSlotPattern;
    // 예: [real(2048²,ASTC), real(2048²,ASTC), dummy(1x1,RGBA)]

    // 스칼라 프로퍼티 패턴
    public List<ScalarEntry> scalarPattern;
}

public struct SlotState
{
    public int slotIndex;           // 셰이더 프로퍼티 선언 순서
    public string propertyName;     // "_NormalMap"
    public bool isRealTexture;      // true=실제 텍스처, false=null/default
    public TextureSignature signature;  // 실제 텍스처면 해상도/포맷
    // isRealTexture == false면 RenderDoc에서 1x1 white로 보임
}

// variant 판별 로직
public static bool IsRealTexture(Material mat, int propertyNameId)
```

```
{
    var tex = mat.GetTexture(propertyNameId);
    if (tex == null) return false;

    // Unity 기본 텍스처 (white, black, gray, bump 등)는 dummy 취급
    if (tex.width <= 4 && tex.height <= 4) return false;

    // 이름으로도 체크 (안전장치)
    var name = tex.name;
    if (name == "unity_default" || name == "" || name == "UnityWhite"
        || name == "UnityBlack" || name == "UnityNormalMap")
        return false;

    return true;
}
```

Variant 매칭 워크플로우:

```
RenderDoc에서 텍스처 슬롯 테이블 복사:
  Slot 0: 2048x2048 ASTC_6x6   (진짜 텍스처)
  Slot 1: 2048x2048 ASTC_6x6   (진짜 텍스처)
  Slot 2: 1x1 RGBA32            (dummy)

UGDB 파서가 슬롯 패턴 추출:
  → [real(2048²,ASTC), real(2048²,ASTC), dummy]

variantBySlotPattern 인덱스에서 검색:
  → Custom/CharacterPBR Variant [_NORMALMAP] 매칭
     (slot 0: _MainTex=real, slot 1: _NormalMap=real, slot 2: _EmissionMap=dummy)

  → _EMISSION 키워드는 OFF 상태 (slot 2가 dummy이므로)

결과:
  Shader: Custom/CharacterPBR
  Active Keywords: _NORMALMAP (ON), _EMISSION (OFF)
  이 variant를 사용하는 머티리얼 12개:
  ├ M_Hero_Cape
  ├ M_NPC_Villager
  └ ... (10개 더)
```

1-4. 복합 매칭 (DrawCall Match)

여러 키를 동시에 입력하면 가중치 스코어로 매칭:

```
public class DrawCallMatcher
```

```
{
    const float WEIGHT_GEOMETRY = 0.30f;  // vtx+idx 일치
    const float WEIGHT_TEXTURE  = 0.25f;  // 텍스처 슬롯 시그니처
    const float WEIGHT_VARIANT  = 0.15f;  // variant 슬롯 패턴 (real/dummy)
    const float WEIGHT_SCALAR   = 0.20f;  // CB 값 일치
    const float WEIGHT_SHADER   = 0.10f;  // 셰이더 키워드

    // 90~100: HIGH (거의 확정)
    // 70~89:  MEDIUM (높은 확률)
    // 50~69:  LOW (후보)
}
```

1-5. 산출물

- `SceneSnapshot.cs` **— 씬 기반 풀체인 수집**
- `VariantTracker.cs` **— variant별 슬롯 패턴 수집 + real/dummy 판별**
- `LookupEngine.cs` **— 5개 인덱스 빌드 + 개별/복합 검색**
- `DrawCallMatcher.cs` **— 복합 매칭 스코어링 (variant 가중치 포함)**
- `SnapshotData.cs` **— 데이터 클래스**
- `SnapshotStore.cs` **— JSON 저장/로드**

Phase 2 — Clipboard Parser + UI (2~3주)

목표: RenderDoc에서 복사한 텍스트를 자동 판별하고 검색 결과를 보여주는 에디터 윈도우

2-1. RenderDoc 복사 포맷 분석

RenderDoc에서 복사되는 텍스트 포맷은 패널마다 다르다. 파서가 인식해야 할 주요 패턴:

```
=== 텍스처 관련 ===

패턴 A: Resource 패널 복사
  "Texture2D 2048x2048 12 mips - BC7_UNORM"
  "Texture2D 512x512 10 mips - ASTC_6x6"
  매칭 regex: /(\d+)x(\d+)\s+(\d+)\s*mips?\s*[--]\s*(\w+)/

패턴 B: Pipeline State 텍스처 슬롯 복사 (탭 구분)
  "0\tTexture2D\t2048x2048\tBC7_UNORM\t12 mips"
  매칭 regex: /(\d+)\t\w+\t(\d+)x(\d+)\t(\w+)\t(\d+)/
```

패턴 C: 텍스처 해상도만
  "2048x2048"
  "1920x1080"
  매칭 regex: /^(\d+)x(\d+)$/

=== 드로우콜 관련 ===

패턴 D: Event Browser 복사
  "DrawIndexed(34560, 1, 0, 0, 0)"
  "DrawIndexed(34560)"
  매칭 regex: /DrawIndexed\((\d+)/

패턴 E: API Inspector 복사
  "DrawIndexed()
     IndexCount: 34560
     InstanceCount: 1
     StartIndexLocation: 0"
  매칭 regex: /IndexCount:\s*(\d+)/

패턴 F: Draw call 이름
  "Draw(1200)"
  매칭 regex: /Draw\((\d+)\)/

=== Constant Buffer 관련 ===

패턴 G: CB 구조체 뷰 복사
  "float4 1.000 0.950 0.900 1.000
   float   0.730
   float   1.000"
  매칭 regex: /float4?\s+([\d.]+(?:\s+[\d.]+)*)/

패턴 H: CB 오프셋 뷰 복사
  "   [0]: 1, 0.95, 0.9, 1
    [16]: 0.73
    [20]: 1"
  매칭 regex: /\[(\d+)\]:\s*([\d.,\s]+)/

패턴 I: CB raw hex 복사
  "00000000: 3F800000 3F733333 3F666666 3F800000"
  매칭: hex → float 변환

=== 셰이더 관련 ===

패턴 J: 셰이더 소스/디스어셈블리에서 키워드
  "#pragma multi_compile _NORMALMAP _EMISSION"
  "Keywords: _NORMALMAP _EMISSION"
  매칭 regex: /_[A-Z][A-Z_0-9]+/g (연속된 대문자 키워드 패턴)

```
패턴 K: 셰이더 이름 (Unity에서 RenderDoc에 넘어간 경우)
  "Custom/CharacterPBR"
  매칭 regex: /\w+\/[\w\/]+/ (슬래시 포함 경로 패턴)

=== 복합 (여러 줄) ===

패턴 L: Pipeline State 텍스처 테이블 전체
  여러 줄의 텍스처 슬롯 정보가 한번에 복사됨
  → 각 줄을 개별 파싱하여 복합 매칭
```

2-2. Clipboard Parser 설계

```
public class ClipboardParser
{
    // 메인 파싱 함수: 텍스트를 받아서 ParseResult 반환
    public ParseResult Parse(string clipboardText)
    {
        var result = new ParseResult();

        // 1. 줄 단위로 분리
        var lines = clipboardText.Split('\n');

        // 2. 각 줄에 대해 모든 패턴 시도 (우선순위 순)
        foreach (var line in lines)
        {
            if (TryParseDrawCall(line, result)) continue;     // 패턴 D,E,F
            if (TryParseTexture(line, result)) continue;      // 패턴 A,B,C
            if (TryParseCBStruct(line, result)) continue;     // 패턴 G
            if (TryParseCBOffset(line, result)) continue;     // 패턴 H
            if (TryParseCBHex(line, result)) continue;        // 패턴 I
            if (TryParseKeywords(line, result)) continue;     // 패턴 J
            if (TryParseShaderName(line, result)) continue;   // 패턴 K
        }

        // 3. 추출된 키 조합으로 검색 타입 결정
        result.queryType = DetermineQueryType(result);
        return result;
    }
}

public class ParseResult
{
    public QueryType queryType;  // Auto-determined
```

```
    // 추출된 키들 (있는 것만 채워짐)
    public int? indexCount;
    public int? vertexCount;
    public List<TextureSignature> textures = new();
    public List<float> cbFloats = new();
    public List<Vector4> cbVectors = new();
    public Dictionary<int, float> cbOffsetValues = new();  // offset → value
    public List<string> shaderKeywords = new();
    public string shaderName;
}

public enum QueryType
{
    Texture,        // 텍스처 정보만 추출됨
    Geometry,       // DrawIndexed 정보만 추출됨
    ConstantBuffer, // CB 값만 추출됨
    Shader,         // 셰이더 이름/키워드만 추출됨
    Composite       // 여러 종류가 동시에 추출됨 → 복합 매칭
}
```

2-3. 선행 작업: RenderDoc 복사 포맷 샘플 수집

파서의 정확도는 실제 복사 포맷을 얼마나 잘 커버하느냐에 달려있다. 개발 착수 전 다음 작업을 먼저 수행:

```
1. RenderDoc 1.26+ (DX11) 에서 캡처
2. 다음 패널에서 각각 복사해서 텍스트 파일로 저장:
   - Event Browser: 드로우콜 우클릭 복사
   - Pipeline State > VS/PS Input: 텍스처 슬롯 행 복사
   - Pipeline State > VS/PS Input: 텍스처 슬롯 테이블 전체 복사
   - Pipeline State > CB 뷰: 구조체 뷰 복사
   - Pipeline State > CB 뷰: 오프셋 뷰 복사
   - Pipeline State > CB 뷰: raw hex 복사
   - Texture Viewer: 리소스 정보 복사
   - Resource Inspector: 리소스 상세 복사
   - Shader Viewer: 디스어셈블리 일부 복사
   - API Inspector: 드로우콜 파라미터 복사
3. Vulkan 캡처에서도 동일 반복
4. 수집된 샘플을 기반으로 regex 패턴 확정
```

이 샘플을 `Editor/UGDB/Tests/RenderDocSamples/` **에 보관하고 유닛 테스트 기반으로 파서를 검증한다.**

2-4. 에디터 윈도우 레이아웃

```
┌──────────────────────────────────────────────────────────────┐
│  UGDB Lookup                          [📸 Snap] [💾 ▾]       │
│  Snapshot: MainStage (234 renderers, 128 textures)           │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  RenderDoc에서 복사한 내용을 붙여넣기:                        │
│   ┌──────────────────────────────────────────────────────┐   │
│   │ DrawIndexed(34560, 1, 0, 0, 0)                       │   │
│   │                                                      │   │
│   │                                                      │   │
│   └──────────────────────────────────────────────────────┘   │
│  감지됨: DrawCall (indexCount: 34560)          [🔍 검색]      │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│  Results (1 match)                                           │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │ Character_Hero                                       │   │
│   │ /MainStage/Characters/Character_Hero                 │   │
│   │ Mesh: SM_Hero_Body (vtx:12450 idx:34560)             │   │
│   │                                                      │   │
│   │ Material: M_Hero_Body                                │   │
│   │ Shader: Custom/CharacterPBR                          │   │
│   │ Keywords: _NORMALMAP, _EMISSION                      │   │
│   │                                                      │   │
│   │ Textures:                                            │   │
│   │   [0] _MainTex → T_Hero_Body_D.png (2048² ASTC)      │   │
│   │   [1] _NormalMap → T_Hero_Body_N.png (2048² ASTC)    │   │
│   │   [2] _EmissionMap → T_Hero_Body_E.png (512² ASTC)   │   │
│   │                                                      │   │
│   │ CB Layout (추정):                                    │   │
│   │   [0] _Color = (1.0, 0.95, 0.9, 1.0)                 │   │
│   │   [16] _Metallic = 0.73                              │   │
│   │   [20] _BumpScale = 1.0                              │   │
│   │                                                      │   │
│   │ [🔍 Ping Asset] [🎯 Select GO] [📋 Copy Path]        │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│  ℹ️ Frame 1234 | 234 renderers | 128 textures (47.2 MB)      │
└──────────────────────────────────────────────────────────────┘
```

복합 붙여넣기 시:

```
┌──────────────────────────────────────────────────────────────┐
│  RenderDoc에서 복사한 내용을 붙여넣기:                        │
```

```
│  ┌──────────────────────────────────────────────────┐  │
│  │ 0  Texture2D  2048x2048  BC7_UNORM  12 mips       │  │
│  │ 1  Texture2D  2048x2048  BC7_UNORM  12 mips       │  │
│  │ 2  Texture2D  512x512    BC7_UNORM  10 mips       │  │
│  │ 3  Texture2D  4096x4096  D32_FLOAT  1 mips        │  │
│  └──────────────────────────────────────────────────┘  │
│  감지됨: Texture Slots ×4                    [🔍 검색]  │
│                                                         │
├─────────────────────────────────────────────────────────┤
│  Slot Mapping:                                          │
│                                                         │
│  Slot 0 (2048² BC7) → T_Hero_Body_D.png (_MainTex)      │
│  Slot 1 (2048² BC7) → T_Hero_Body_N.png (_NormalMap)    │
│  Slot 2 (512² BC7)  → T_Hero_Body_E.png (_EmissionMap)  │
│  Slot 3 (4096² D32) → _ShadowMapTexture (글로벌)        │
│                                                         │
│  Best match: M_Hero_Body (Custom/CharacterPBR) [95%]    │
│  [🔍 Ping Material] [🎯 Select GO]                      │
└─────────────────────────────────────────────────────────┘
```

2-5. 핵심 UI 기능

기능	설명
Ctrl+V 붙여넣기	텍스트 영역에 포커스 상태에서 붙여넣기 → 자동 파싱 + 검색
자동 감지 라벨	파싱 결과를 "감지됨: Texture / DrawCall / CB / Composite" 로 표시
결과 카드	매칭된 에셋의 상세 정보 + 사용처 + CB 레이아웃
Ping Asset	Project 윈도우에서 해당 에셋 하이라이트
Select GO	Hierarchy에서 해당 GameObject 선택 (Play 모드 중)
Copy Path	에셋 경로를 클립보드에 복사
신뢰도 표시	복합 매칭 시 점수 + 등급 (✅🟡🔴)
히스토리	이전 검색 결과 목록 (뒤로가기)
수동 검색 폴백	파서가 인식 못한 경우를 위한 수동 입력 탭

2-6. 산출물

- `ClipboardParser.cs` — RenderDoc 텍스트 자동 판별 + 키 추출

- `ClipboardPatterns.cs` **— regex 패턴 정의 (분리하여 유지보수 용이)**
- `UGDBWindow.cs` **— 메인 에디터 윈도우**
- `ResultsPanel.cs` **— 검색 결과 카드 렌더링**
- `Editor/UGDB/Tests/` **— 파서 유닛 테스트 + RenderDoc 샘플 데이터**

Phase 3 — RenderDoc 동시 캡처 (1~2주)

목표: Snap 시 RenderDoc 캡처를 동시에 트리거하여 같은 프레임의 .rdc 생성

3-1. 동시 캡처 플로우

```
[📸 Snap] 클릭
  ├── RenderDoc 캡처 트리거 → .rdc 파일 생성
  ├── WaitForEndOfFrame
  ├── SceneSnapshot 수집 → 인덱스 빌드
  └── 세션 저장:
      UGDBCaptures/
      └── 20260320_143022/
          ├── snapshot.json   ← 인덱싱 데이터
          ├── capture.rdc     ← RenderDoc 캡처
          └── metadata.json   ← Unity 버전, 해상도 등
```

3-2. RenderDoc 연동

```
public static class RenderDocBridge
{
    public static bool IsAvailable()
        => RenderDoc.IsLoaded() && RenderDoc.IsSupported();

    public static void TriggerCapture(EditorWindow gameView)
        => gameView.SendEvent(
            EditorGUIUtility.CommandEvent("RenderDocCapture"));
}
```

3-3. 산출물

- `RenderDocBridge.cs` **— 캡처 트리거 + .rdc 파일 관리**
- `CaptureCoordinator.cs` **— Snap + RenderDoc 동기화**

- `SessionManager.cs` — 세션(snapshot.json + .rdc) 관리

Phase 4 — pyrenderdoc 자동 매칭 (3~4주, 선택)

목표: .rdc를 파싱하여 전체 드로우콜 목록을 추출하고, 수동 붙여넣기 없이 자동 매칭

4-1. 자동 매칭 플로우

```
[Auto-Match] 클릭
  ├── pyrenderdoc로 .rdc 파싱 → 전체 드로우콜의 GPU 바인딩 (JSON)
  ├── snapshot.json의 인덱스와 자동 매칭
  └── 결과: 모든 드로우콜에 Unity 라벨 자동 부여

  Draw #47 → Character_Hero / M_Hero_Body (95%)
  Draw #48 → Character_Hero / M_Hero_Cape (90%)
  Draw #49 → (shadow pass, unmatched)
  Draw #50 → Ground / M_Terrain (85%)
  ...
```

4-2. 산출물

- `extract_rdc_bindings.py` — pyrenderdoc 파싱 스크립트
- `AutoMatcher.cs` — .rdc JSON + snapshot 자동 매칭
- `AutoMatchPanel.cs` — 전체 드로우콜 매칭 결과 UI

---

5. 기술적 고려사항

5-1. 리스크

문제	난이도	대응 방안
RenderDoc 복사 포맷이 버전마다 다름	높음	선행 샘플 수집 + regex 패턴을 외부 설정으로 분리 + 유닛 테스트
셰이더 **variant** 수가 많음	중간	키워드 조합 + 텍스처 슬롯 real/dummy 패턴으로 variant 특정, 씬에서 실제 사용 중인 variant만 인덱싱

키워드 OFF인데 텍스처가 세팅된 경우	중간	아티스트가 키워드 OFF인데 텍스처를 남겨둔 경우 → 해상도 4x4 이하를 dummy 기준으로 판별, 이름 기반 폴백 체크
같은 해상도/포맷 텍스처 다수	중간	단독 매칭은 후보 목록 표시, 복합 매칭(vtx/idx + CB + variant)으로 좁히기
동일 메시/머티리얼 N개 오브젝트	높음	"매칭 그룹"으로 묶어 표시, Instancing이면 instanceCount로 구분
CB offset ↔ 프로퍼티 매핑	중간	셰이더 프로퍼티 선언 순서로 추정 + 값 일치 확인, variant에 따라 CB 레이아웃이 달라질 수 있음
float 정밀도	낮음	epsilon 비교 (기본 0.001)
DX11 vs Vulkan 포맷 차이	중간	포맷 이름 매핑 테이블 (BC7_UNORM ↔ ASTC_6x6 등)

5-2. 한계

- 셰이더 내부 분기에 따른 텍스처 접근 패턴은 추적 불가 (GPU 디버거 영역)
- 픽셀 단위 "이 색상이 어떤 텍스처에서 왔는지"는 불가 (RenderDoc에서 직접 확인)
- CommandBuffer 내부에서 동적으로 바인딩하는 텍스처는 글로벌 텍스처 DB로 추정만 가능
- CB offset은 추정값 — 셰이더 컴파일러가 패딩/재배치할 수 있으므로 100% 보장 불가

---

6. 일정 요약

```
Week 0      ▒▒▒
```

```
Assets/
└── Editor/
    └── UGDB/
        ├── Core/
        │   ├── SceneSnapshot.cs          ← 씬 기반 풀체인 수집
        │   ├── VariantTracker.cs         ← variant 슬롯 패턴 + real/dummy 판별
        │   ├── LookupEngine.cs           ← 5개 인덱스 + 검색
        │   ├── DrawCallMatcher.cs        ← 복합 매칭 스코어링
        │   ├── SnapshotData.cs           ← 데이터 클래스
        │   └── SnapshotStore.cs          ← JSON 저장/로드
        ├── Parser/
        │   ├── ClipboardParser.cs        ← 텍스트 자동 판별 + 키 추출
        │   └── ClipboardPatterns.cs      ← regex 패턴 정의 (외부 수정 용이)
        ├── RenderDoc/
        │   ├── RenderDocBridge.cs        ← 캡처 트리거
        │   ├── CaptureCoordinator.cs     ← Snap + RenderDoc 동기화
        │   └── SessionManager.cs         ← 세션 관리
        ├── UI/
        │   ├── UGDBWindow.cs             ← 메인 에디터 윈도우
        │   ├── PasteArea.cs              ← 붙여넣기 영역 + 감지 라벨
        │   ├── ResultsPanel.cs           ← 결과 카드 UI
        │   ├── ManualSearchPanel.cs      ← 수동 검색 폴백 UI
        │   └── AutoMatchPanel.cs         ← Phase 4 자동 매칭 UI
        ├── Tests/
        │   ├── ClipboardParserTests.cs   ← 파서 유닛 테스트
        │   └── RenderDocSamples/         ← 실제 복사 텍스트 샘플
        └── Python/
            └── extract_rdc_bindings.py   ← Phase 4 pyrenderdoc 파싱
```

---

8. 확장 가능성

- **텍스처 메모리 리포트:** 스냅샷 데이터로 "프레임 내 텍스처 메모리 TOP 20" 즉시 생성
- **중복 텍스처 감지:** 같은 텍스처가 여러 머티리얼에서 다른 이름으로 참조되는 패턴
- **셰이더 variant 매트릭스:** 씬 내 키워드 조합 시각화
- **스냅샷 diff:** 두 스냅샷 간 머티리얼/텍스처 변경 비교
- **Asset Store 제품화:** 에디터 패키지로 정리하여 출시
- **커스텀 파서 패턴 추가:** Snapdragon Profiler, Nsight Graphics 등 다른 GPU 디버거 복사 포맷 지원